

Forced Verdict Gates, Not Graph Sophistication: A Multi-Framework Benchmark of AI Audit Judgement on Real-World Scandal Cases

Author: Majid Mumtaz (ACCA, CIA, ACA)

Date: 2026-07-08

Status: Discussion paper draft. Comments welcome.

Abstract

We benchmark four multi-agent orchestration frameworks (plain-Python Blackboard, CrewAI, OpenAI Swarm, LangGraph) on a shared audit-judgement task: four AI agents embodying canonical audit personas (Engagement Partner, Audit Senior, Skeptic Reviewer, Independence Checker) must each form an audit opinion on four real-world corporate scandals (Enron 2001, Wirecard 2020, SVB 2023, Tesco 2014), using only the public information available *before* each scandal broke. Each framework's output is scored against hidden ground truth on four axes — judgement accuracy, evidence rigour (red-flag recall), reasoning quality (LLM-judged), and calibration — plus a manual framework-ergonomics axis.

The lowest-abstraction framework (plain Python with a Blackboard pattern) outperformed the three production frameworks (15.33 vs 13.84, 12.94, 5.00 out of 20 on auto-scored axes). We initially hypothesised the differentiator was a *forced verdict gate* — an independence-checker node that rejects unsupported opinions and forces rework. A deconfounding experiment adding the gate to LangGraph did not confirm this: the gated variant (12.64–13.01) did not recover plain_python's score, ruling out the gate as the sole driver and leaving the Blackboard's input-digest reactivity as the leading alternative explanation. Evidence-surfacing and opinion-formation emerged as decoupled capabilities: the framework with the highest red-flag recall (Swarm, 4.44/5, including 100% on one case) had the lowest judgement accuracy (1.50/5) and refused to commit to any verdict. Agents were systematically over-cautious relative to the real auditors who failed (0 of 16 runs issued a "Clean" opinion; the real auditors issued Clean on all four), and mostly correctly so given the evidence constraint. The paper was independently peer-reviewed by a second LangGraph multi-agent system (three reviewer personas + editor, with a conditional reconsideration loop); the peer review correctly identified the gate-framework confound that the deconfounding experiment confirmed. We discuss implications for multi-agent framework design, LLM-as-auditor research, and the audit profession's structural under-caution problem.

Keywords: multi-agent systems, audit judgement, LLM agents, framework benchmark, professional skepticism, calibration

1. Introduction

The audit profession has a recurring failure mode: auditors issue clean opinions on financial statements that are later revealed to be materially misstated. Enron (2001), Wirecard (2020), Silicon Valley Bank (2023), and Tesco (2014) are four instances separated by two decades and three jurisdictions, each involving a different Big Four auditor, and each ending in a clean or unqualified opinion that was indefensible in hindsight. The structural causes are well-documented: client-pays economics create auditor dependence (Watts and Zimmerman, 1983; DeAngelo, 1981), long auditor tenure erodes skepticism (Johnson et al., 2002; Carey and Simnett, 2006), and professional standards permit auditors to defer to management representations when independent evidence is costly to obtain (ISA 500, IAASB, 2024).

A natural question for the AI-as-auditor research programme is whether LLM-based agent teams, unburdened by client-retention economics and capable of systematic evidence-citation, would avoid this failure mode. Multi-agent frameworks — CrewAI, OpenAI Swarm, LangGraph, and the Blackboard pattern that predates them all — offer different orchestration models for teaming AI agents on professional judgement tasks. But which framework features actually drive *audit judgement quality* (as distinct from reasoning quality, evidence recall, or conversational fluency) is an open question. Framework comparisons in the literature typically evaluate on coding, retrieval, or customer-service tasks (Liu et al., 2023; Mialon et al., 2023; Jimenez et al., 2023); none to our knowledge evaluate on audit opinion formation against historical ground truth.

This paper makes three contributions:

1. **A benchmark design for audit-judgement tasks.** Four real scandal cases, each with agent-visible pre-scandal evidence and agent-invisible ground truth; four canonical audit personas held constant across frameworks; a four-axis scorer (judgement accuracy, evidence rigour, reasoning quality, calibration) that separates evidence-surfacing from opinion-formation.
2. **An empirical comparison of four frameworks on that benchmark.** 16 runs (4 frameworks × 4 cases) using a single LLM (glm-5.2:cloud via Ollama Cloud) to isolate the framework effect from the model effect.
3. **A hypothesis about which framework feature matters, tested and revised through deconfounding.** The initial differentiator was not graph sophistication (LangGraph's conditional revisit loop fired on 3 of 4 cases but did not flip any wrong verdict to correct) or native tool-calling (Swarm's function calls worked perfectly but its handoff chain never terminated at an opinion). We hypothesised a *forced verdict gate* — a node whose only job is to reject unsupported opinions and force rework — as the driver. A deconfounding

experiment adding the gate to LangGraph did not confirm this; the gate alone did not recover plain_python's performance. We argue the leading alternative explanation is the Blackboard pattern's input-digest reactivity, which automatically threads all upstream writes to downstream agents without manual state-field plumbing — but we cannot distinguish this from run-to-run variance at $n=1$ per cell. The paper was independently peer-reviewed by a second LangGraph multi-agent system, whose reviewers correctly identified the gate-framework confound that the deconfounding experiment confirmed; this peer-review process is itself part of the methodology (§3.4).

The paper is structured as follows. Section 2 reviews related work. Section 3 describes the benchmark design. Section 4 presents results. Section 5 discusses the six findings. Section 6 addresses limitations. Section 7 concludes.

2. Related work

Multi-agent frameworks. The Blackboard pattern (Nii, 1986; Corkill, 1991) is the oldest architecture for coordinating multiple agents on a shared problem-space; it predates LLMs by decades and has been reapplied to LLM agent teams in recent work. CrewAI (CrewAI, 2025) provides role-based agents with sequential or hierarchical process orchestration and automatic task-context threading. OpenAI Swarm (OpenAI, 2024) is an educational framework built around agent-to-agent handoffs and function-call tools. LangGraph (LangChain, 2025) offers a StateGraph with conditional edges as a low-level orchestration primitive, supporting loops, human-in-the-loop, and durable execution. Each framework exposes a different theory of how agents should coordinate; none publishes benchmarks on professional judgement tasks.

LLM-as-auditor. Prior work has explored LLMs for audit-relevant sub-tasks: anomaly detection in journal entries (Schreyer et al., 2018). To our knowledge no prior work benchmarks *multi-agent audit opinion formation* against historical scandal ground truth.

Audit judgement literature. The profession's own research on judgement quality predates AI: the "audit judgement" literature studies how auditors form opinions under evidence constraint, with findings on overconfidence (Han et al., 2011; Libby, 1995), anchoring on prior-year comfort (Butt and Campbell, 1989), and the effect of engagement quality review on opinion quality (Schneider and Messier, 2007; Epps and Messier, 2007; PCAOB AS 1210). Our investigation of the verdict gate — an independence-checker that rejects unsupported opinions and forces rework — maps onto this literature, though the deconfounding experiment (§4.5) showed the gate alone does not explain the effect, leaving the state-propagation mechanism as a co-determinant (§5.1).

Framework benchmarks. Existing agent benchmarks (AgentBench, Liu et al., 2023; GAIA, Mialon et al., 2023; SWE-bench, Jimenez et al., 2023) evaluate coding, retrieval, and reasoning tasks.

None evaluate professional judgement under evidence constraint against historical ground truth. Our benchmark fills this gap for the audit domain.

Note on citations: Citations in this draft use author-year in-text format. All references are real published works or authoritative documentation. Three references on LLM-as-auditor sub-tasks (going-concern prediction, audit report analysis) were removed because they could not be verified in published form; the author will add further verified LLM-as-auditor citations in the next revision.

3. Methodology

3.1 Benchmark design

The benchmark consists of four components held constant across all framework implementations: cases, personas, output contract, and scorer.

3.1.1 Cases

Four real-world corporate scandals were selected to span two decades, three jurisdictions (US, Germany, UK), and four different auditors (Arthur Andersen, EY, KPMG, PwC):

Case	Entity	Auditor	FY audited	Real opinion	Hindsight-correct	Acceptable ratings
Enron 2001	Enron Corp	Arthur Andersen	FY2000	Clean	Adverse	{Adverse, Qualified}
Wirecard 2020	Wirecard AG	EY Munich	FY2019	Clean	Adverse	{Adverse, Disclaimer}
SVB 2023	SVB Financial	KPMG	FY2022	Clean	Qualified	{Qualified, Adverse}
Tesco 2014	Tesco PLC	PwC	FY2013/14	Clean	Qualified	{Qualified, Disclaimer}

Each case directory contains two partitions:

- **evidence/** — agent-visible. A curated subset of the public information available *before* the scandal broke: 10-K excerpts, audit opinion text, management representations, analyst notes, regulatory filings. We erred toward what was in the financial statements plus one or two analyst/press items. A real auditor would have had more (working papers, management access). The benchmark tests judgement under *evidence constraint*, not full audit conditions.
- **ground_truth.json** — agent-invisible, scored-against. Contains the scandal summary, the correct opinion in hindsight, a list of 8–9 key red flags that should have been raised, and metadata on evidence sufficiency and independence concerns.

The agent-visible / agent-invisible split is the methodological core: it tests whether agents can reason *forward* from limited evidence to the right concern, not pattern-match a famous scandal name to a known outcome.

3.1.2 Personas

Four audit personas, constant across all frameworks:

Persona	Role	Function
EngagementPartner	Sets scope, materiality; gives final opinion	Weighs evidence and red flags; commits to a rating
AuditSenior	Substantive testing lead	Enumerates red flags, each citing a specific evidence item
SkepticReviewer	Adversarial professional skepticism	Argues against the prevailing view; hunts fraud indicators
IndependenceChecker	Audit independence & quality gate	Verifies every claim in the draft opinion is backed by cited evidence; rejects if not

Persona definitions (system prompt, goal, constraints) are specified in `audit_bench/common.py` and imported identically by every framework module. This isolates the framework orchestration effect from the persona-prompt effect.

3.1.3 Output contract

Every framework's final output must contain an <OPINION> JSON block with four fields:

```
{ "rating": "Clean|Qualified|Adverse|Disclaimer|Unable",  
  "reasoning": "<one-paragraph justification citing evidence>",  
  "red_flags": ["<flag with citation>", "..."],  
  "unable_to_determine": true|false }
```

The five-point rating scale maps to the standard audit opinion taxonomy (with "Unable" added for cases where the agent team explicitly declines to opine due to evidence insufficiency). The contract is enforced by a regex parser (`common.py:parse_opinion`); frameworks that fail to emit parseable JSON default to `rating="Unable"`, `unable_to_determine=true`.

3.1.4 Scorer

Four scored axes (0–5 each) plus one manual axis:

4. **Judgement accuracy.** Exact match against `ground_truth.acceptable_ratings` (a set, since audit opinions are not uniquely determined) → 5. Adjacent (one step away on the ordinal scale `Clean`→`Qualified`→`Adverse/Disclaimer`→`Unable`) → 3. Wrong → 0.
5. **Evidence rigour.** Fraction of `ground_truth.key_red_flags` cited by the agent (substring + keyword-category match), scaled ×5.

6. **Reasoning quality.** LLM-judged rubric score 0–5, single call to glm-5.2:cloud per run. The rubric (in `scorer.py: JUDGE_RUBRIC`) rates evidence citation specificity, counter-evidence consideration, and identification of structural failure modes.
7. **Calibration.** Whether the agent's `unable_to_determine` flag matches the ground-truth evidence sufficiency. "Calibrated" → 5. "Overconfident" (committed when evidence was insufficient) → 2. "Underconfident" (declined when evidence was sufficient) → 2.
8. **Framework ergonomics.** Manual, judged after building all four implementations on LOC, debuggability, boilerplate-to-logic ratio, and operational surprises.

3.2 Framework implementations

Each framework implements a common interface: `run_case(case_dir, model, logger)` → `dict` returning the structured opinion plus a full transcript. Frameworks differ in orchestration model; personas and output contract are identical.

3.2.1 plain_python (Blackboard baseline)

Reuses the Blackboard/Cook/Kitchen pattern: a shared Blackboard data structure that cooks (agents) read from and write to, with reactive execution via input digests (a cook re-runs only if its inputs changed since last run). Four cooks run in fixed order (Senior → Skeptic → Partner → Independence), up to 3 rounds. The IndependenceChecker's rejection triggers a loop back to the Senior. 112 LOC.

3.2.2 CrewAI

Four Agent objects (one per persona) with backstory matching the persona constraints. Four Task objects chained via `Process.sequential`; each task's output auto-threads as context to the next. `LLM(model="glm-5.2:cloud", base_url="https://ollama.com/v1", api_key=...)` — the bare model name is load-bearing: the `openai/` prefix triggers LiteLLM provider routing which fails without `litellm` installed. 111 LOC.

3.2.3 OpenAI Swarm

Four Agent objects with explicit `transfer_to_*` handoff functions (Senior → Skeptic → Partner → Independence) and four function-call tools (`cite_red_flag`, `record_skeptic_point`, `set_draft_opinion`, `signoff_independence`). `Swarm(client=OpenAI(api_key=..., base_url="https://ollama.com/v1"))`. The client's `run()` method uses `context_variables=` (not `context=`) and has no `max_tokens` parameter — both are load-bearing API quirks. 133 LOC.

3.2.4 LangGraph

A `StateGraph` over a `TypedDict` state schema with fields for evidence, red flags, skeptic objections, draft opinion, and independence decision. Nodes: `read_evidence` → `senior` → `skeptic` → `partner` → `independence`. The differentiator is a *conditional edge* from skeptic back to senior when the skeptic raises objections the senior did not address, capped at `MAX_SKEPTIC_REVISITS=2`. `ChatOpenAI(model=..., base_url="https://ollama.com/v1", api_key=...)`. 177 LOC.

3.3 Experimental setup

- **Single LLM** across all frameworks: `glm-5.2:cloud` via Ollama Cloud's OpenAI-compatible endpoint. Isolates the framework effect from the model effect.
- **Sequential calls** (no parallelism), 180s timeout per call. Reasoning models require high `max_tokens` (8192) and long timeouts or they return empty content.
- **Single LLM judge** for reasoning quality: `glm-5.2:cloud` with a tight rubric. Known self-preference risk; mitigated by the rubric's specificity and the single-call design. The scorer's reasoning-quality axis uses a sentinel value of -1 when the judge LLM call itself fails; this is included in some aggregate means in §4.1 (notably Swarm's 3.50, which is the mean of {5, 5, 5, -1}) — a known scoring artefact flagged in §6.
- **Native tool-calling verified pre-benchmark.** A Phase 0 probe (`_probe_tool_call.py`) confirmed both `glm-5.2:cloud` and `deepseek-v4-pro` emit valid OpenAI-style `tool_calls` via Ollama Cloud (3.7s and 1.7s respectively, with parseable JSON arguments). This permitted Swarm's function-call design and CrewAI's LLM abstraction to operate against the custom endpoint.
- **16 runs total** (4 frameworks × 4 cases × 1 model), executed sequentially in 64 minutes wall-clock.
- **Deconfounding runs** (§4.5): 8 additional runs (`langgraphgate` × 4 cases, `langgraphgate3` × 4 cases), 26 min + 8.4 min respectively.

3.4 Peer review as methodology

The paper was independently peer-reviewed by a second LangGraph multi-agent system (`peer_review.py`) after the initial results were written up but before the deconfounding experiment. The peer-review graph consists of four personas — MethodologyReviewer (audit-judgement literature), EmpiricalReviewer (agent benchmarks), ConceptualReviewer (multi-agent systems theory), and Editor (synthesis + decision) — with a conditional reconsideration edge: if the editor detects a contradiction between two reviewers on a material point, the flagged reviewer is sent back for reconsideration (cap 1 per reviewer). The graph fired once (ConceptualReviewer reconsidered) across the run, producing a `major_revisions` decision with 11 required revisions.

The peer review is part of the experimental methodology, not a separate quality-assurance step: it surfaced the gate-framework confound (independently flagged by all three reviewers) that the deconfounding experiment in §4.5 then confirmed. The peer-review output is in `audit_bench/runs/peer_review_1783445236/peer_review.md`; the deconfounding response is in `audit_bench/DECONFOUNDING_REPORT.md`. Both are primary source material for the paper's argument that the peer-review process itself — run as a multi-agent system — can catch confounds the authors missed.

4. Results

4.1 Headline scores

Framework	Judgement accuracy	Evidence rigour	Reasoning quality	Calibration	Framework ergonomics	Total / 25
plain_python	3.25	3.58	4.25	4.25	4.5	15.83
langgraph	2.50	3.59	4.25	3.50	3.5	13.84
swarm	1.50	4.44	3.50†	3.50	3.0	12.94
crewai	1.50	0.00	0.00	3.50	1.5	5.00

†Swarm's reasoning-quality mean of 3.50 is the average of {5, 5, 5, -1} where -1 is a sentinel for a failed judge LLM call on the Wirecard case, not a real score. Excluding the failed call, Swarm's reasoning mean is 5.00 on 3 of 4 cases. This is a known scoring artefact (§6 Limitation 3) and inflates the cross-framework comparison's noise on this axis.

4.2 Per-case detail

Framework	Case	Agent rating	Ground truth	Rating accuracy	Recall	Reasoning /5	Calibration	Elapsed s
plain_python	Enron 2001	Disclaimer	Adverse	wrong	0.67	4	calibrated	338
plain_python	SVB 2023	Qualified	Qualified	exact	0.67	5	calibrated	188
plain_python	Tesco 2014	Adverse	Qualified	adjacent	0.75	5	overconfident	354
plain_python	Wirecard 2020	Disclaimer	Adverse/Disclaimer	exact	0.78	3	calibrated	288
crewai	Enron 2001	Unable	Adverse	adjacent	0.00	0	underconfident	416
crewai	SVB 2023	Unable	Qualified	wrong	0.00	0	underconfident	432
crewai	Tesco 2014	Unable	Qualified	wrong	0.00	0	calibrated	113
crewai	Wirecard 2020	Unable	Adverse/Disclaimer	adjacent	0.00	0	calibrated	221
swarm	Enron 2001	Unable	Adverse	adjacent	0.78	5	underconfident	56
swarm	SVB 2023	Unable	Qualified	wrong	0.89	5	underconfident	73

swarm	Tesco 2014	Unable	Qualified	wrong	1.00	5	calibrated	40
swarm	Wirecard 2020	Unable	Adverse/Disclaimer	adjacent	0.89	—	calibrated	46
langgraph	Enron 2001	Disclaimer	Adverse	wrong	0.78	5	underconfident	693
langgraph	SVB 2023	Unable	Qualified	wrong	0.67	3	underconfident	240
langgraph	Tesco 2014	Qualified	Qualified	exact	0.88	5	calibrated	79
langgraph	Wirecard 2020	Disclaimer	Adverse/Disclaimer	exact	0.56	4	calibrated	74

4.3 LangGraph revisit-loop behaviour

The conditional skeptic→senior edge fired on 3 of 4 cases:

Case	Skeptic revisits	Senior calls	Skeptic calls	Outcome
Enron 2001	2 (hit cap)	2	2	Disclaimer (wrong)
SVB 2023	2 (hit cap)	2	2	Unable (wrong)
Tesco 2014	1	1	1	Qualified (exact)
Wirecard 2020	0 (skeptic conceded)	1	1	Disclaimer (exact)

The loop engaged and surfaced additional red flags (Enron recall 0.78, Tesco recall 0.88) but did not flip any wrong verdict to correct on the two cases that hit the cap.

4.4 Rating distribution

Across all 16 runs: Unable 9, Disclaimer 4, Qualified 2, Adverse 1, Clean 0. (The real auditors issued Clean on all four cases. No AI team replicated the under-caution failure mode.) Note: an earlier draft of this paper reported an incorrect distribution (Disclaimer 6, Unable 5, Qualified 2, Adverse 2 summing to 15); the corrected counts above sum to 16. The error was identified by the peer-review system (§3.4) and is documented here as a transparency point.

4.5 Deconfounding experiment: adding the verdict gate to LangGraph

The peer review (§3.4) flagged that the verdict gate was perfectly confounded with framework identity — only *plainpython* had a gate. To isolate the gate as a variable, we implemented the same gate mechanism (*independence-checker rejection loop, conditional edge back to senior*) inside *LangGraph*, holding everything else identical to `langgraphfw.py`. Two variants were run:

- **langgraph_gate** (cap=2 gate revisits): 4 cases, 26 min wall-clock, run bench_1783447036/

- **langgraph_gate3** (cap=3, matching plainpython's 3-round cap): 4 cases, 8.4 min, run ``bench1783449438/``

Framework	Mean total / 20	Enron	SVB	Tesco	Wirecard
plain_python (gate, cap=3)	15.32	Disclaimer (wrong, 12.33)	Qualified (exact, 18.33)	Adverse (adj, 13.75)	Disclaimer (exact, 16.89)
langgraph (no gate)	13.84	Disclaimer (wrong, 10.89)	Unable (wrong, 8.33)	Qualified (exact, 19.38)	Disclaimer (exact, 16.78)
langgraph_gate3 (gate, cap=3)	13.01	Disclaimer (wrong, 7.22)	Disclaimer (adj, 13.00)	Adverse (adj, 14.38)	Disclaimer (exact, 17.44)
langgraph_gate (gate, cap=2)	12.64	Disclaimer (wrong, 9.89)	Unable (wrong, 10.44)	Adverse (adj, 18.00)	Unable (adj, 12.22)

(Totals here are on the 4 auto-scored axes only, /20; the §4.1 headline table uses /25 including manual ergonomics. The deconfounding comparison uses /20 for apples-to-apples since the manual axis is not available for the new variants.)

Result: Adding the gate to LangGraph did not improve judgement — it went slightly *down* (langgraph 13.84 → langgraphgate 12.64 → *langgraphgate3* 13.01). The cap was not the confound: raising it from 2 to 3 improved the mean only marginally (+0.37, within noise) and the gap to plainpython (15.32) *persists at 2.31 points*. *The gate fired on 3 of 4 cases in both variants but did not flip any wrong verdict to correct in either run. The gate-as-driver hypothesis is not supported; the leading alternative explanation is the Blackboard pattern's input-digest reactivity (§5.1). Full case-by-case detail is in `DECONFOUNDINGREPORT.md`.*

5. Discussion

5.1 Finding 1 (revised) — plain_python won; the verdict gate hypothesis was not confirmed

plain_python (15.33/20 on auto-scored axes) outperformed the three production frameworks (langgraph 13.84, swarm 12.94, crewai 5.00). We initially attributed the win to the **independence-checker rejection loop** (the "verdict gate"): the EngagementPartner emits a draft opinion, the IndependenceChecker inspects it claim-by-claim against cited evidence, and if any claim is unsupported the loop sends the team back to the AuditSenior for another round (max 3). The hypothesis was that the gate creates *forced commitment* — the team must either produce a citeable opinion or be marked "Unable".

The peer review (§3.4) flagged that the gate was perfectly confounded with framework identity: only *plainpython* had a gate. The deconfounding experiment (§4.5) added the same gate mechanism to *LangGraph*. ****The gate did not recover *plainpython*'s performance:**** *langgraphgate* (cap=2) scored 12.64 and *langgraphgate3* (cap=3, matching *plainpython*'s cap) scored 13.01, vs *plainpython*'s 15.32 — a 2.31-point gap that persists with the cap confound ruled out. The gate fired on 3 of 4 cases in both variants but did not flip any wrong verdict to correct. The gate-as-driver hypothesis is not supported by the deconfounding data.

Three alternative explanations remain, none distinguishable at n=1 per cell:

9. **The Blackboard's input-digest reactivity** (the leading candidate, raised by the peer review's ConceptualReviewer). *plainpython*'s cooks automatically see **all* upstream writes via the digest mechanism* — the senior's second pass after IC rejection automatically inherits the partner's prior draft, the skeptic's objections, and the IC's rejection in one snapshot. *langgraphgate3* passes context manually through state fields; if any field is omitted from a node's return, the senior's rework is degraded. This is a real architectural difference, not a prompt difference, and it plausibly explains why the same gate mechanism works in *plain_python* but not in *langgraph*.
10. **Run-to-run LLM variance.** With n=1 per cell, a 2.31-point gap on a 20-point scale is plausibly noise. The paper's own Limitation 1 says differences ≤ 0.5 should not be treated as significant; 2.31 is above that threshold but n=4 still does not support strong claims. A multi-run design (3 runs per cell, n=12) would settle this.
11. **Prompt threading asymmetry.** *plainpython*'s blackboard snapshot exposes the full state to every cook; *langgraph*'s nodes see only the state fields they explicitly read. If the senior's rework in *plainpython* benefits from seeing the partner's prior draft (which is on the blackboard) while *langgraph*'s senior sees only the IC rejection (not the prior partner draft), the information asymmetry could explain the gap. This is testable by adding the partner's prior draft to *langgraph_gate3*'s senior node state field.

The defensible claim is weaker than the original: the verdict gate may be a **necessary but not sufficient** condition (no framework without a gate beat *plainpython*, but adding the gate to *langgraph* did not recover *plainpython*'s score). The gate maps onto a real audit-firm structural feature — engagement quality review (PCAOB AS 1210, ISA 220) — but the mechanism transfer is not as clean as the original Finding 1 claimed: EQR's effectiveness derives from the reviewer's organisational independence and experiential judgement, neither of which a prompt-driven LLM sharing the same model as the agents it checks possesses (ConceptualReviewer's critique). The gate is a forcing function; whether it is also a quality review depends on the checker's detection reliability, which this paper does not evaluate.

Implication for framework design. The deconfounding result suggests that *how* a verdict gate is wired into the framework's control flow matters as much as *whether* one exists. The Blackboard pattern's automatic state propagation may be doing load-bearing work that a manual state-field plumbing approach (*LangGraph*) does not replicate. Framework designers should treat the

verdict gate as a necessary structural feature but should also consider the *state propagation mechanism* — how upstream writes reach downstream agents on rejection-revisit — as a co-determinant of gate effectiveness. This is a hypothesis for future work with a multi-run design.

5.2 Finding 2 — Evidence-surfacing and opinion-formation are decoupled

Swarm achieved the highest red-flag recall in the benchmark (mean 4.44/5, including 100% on Tesco — every ground-truth flag surfaced) but the lowest judgement accuracy (1.50/5) and 0/4 exact ratings. It found every flag and still said "Unable". Inspection of swarm/casetesco2014's transcript shows the final message was "*Red Flag 10: Gross vs. net recognition — no evidence PwC tested offsetting costs*" — the chain was still appending red flags when the run terminated; the partner's opinion-formation step was never reached.

Conversely, plain_python produced the best judgement accuracy (3.25/5, the only exact "Qualified" on SVB) but mid-tier recall (3.58/5). Its independence-checker forced opinion commitment at the cost of not exhausting every possible red flag.

The cross-framework correlation between judgement accuracy and evidence recall, computed on the 4 framework means, is *positive* (Pearson $r \approx +0.37$ with all four frameworks included; $r \approx -0.91$ if crewai is excluded, but $n=3$ cannot support a directional claim and the paper never justified excluding crewai). An earlier draft of this paper claimed the correlation was negative — this was a factual error identified by the peer review (§3.4) and corrected here. The *qualitative* contrast that survives is the within-data-point observation that Swarm (highest recall, lowest accuracy) and plain_python (moderate recall, highest accuracy) occupy opposite corners of the 2D space. This is a case-level observation about two frameworks, not a cross-framework correlation. The architectural choices that make a framework good at *listing* risks are not the choices that make it good at *deciding* on them — but with $n=4$ frameworks this remains an observation, not an established regularity.

Implication for benchmark design. Future audit-judgement benchmarks must score evidence-surfacing and opinion-formation as separate axes (as this one does), not collapse them into a single "audit quality" score. A framework that dominates one axis and fails the other is not half-good — it is unqualified for a different reason than a framework that fails both. The audit profession's own standards (ISA 700, ISA 705) treat the opinion and the evidence basis as separable: a qualified opinion with adequate evidence is a different deliverable than an unqualified opinion with inadequate evidence, and benchmarks should reflect this.

5.3 Finding 3 — Reasoning quality is not the bottleneck; commitment is

The LLM judge awarded 5/5 reasoning to 7 of 16 runs, including runs whose final rating was wrong. The model writes auditor-grade prose: it cites specific footnote numbers, distinguishes HTM from AFS classification under ASC 320, references ISA 530 sampling adequacy, and reasons about materiality thresholds. The failure mode is not inability to reason — it is inability to *commit to a verdict* on the basis of that reasoning.

Exemplar: langgraph/Enron received 5/5 reasoning with 0.78 recall and produced a correctly-argued Disclaimer — but the ground-truth acceptable ratings were {Adverse, Qualified}. The reasoning was sound; the verdict was too cautious. The model knew the evidence was damning and wrote as much, then declined to issue Adverse.

Implication for LLM-as-auditor research. The field's emphasis on reasoning benchmarks (chain-of-thought, audit-step-by-step, structured reasoning traces) may be misdirected for the opinion-formation task. The bottleneck is the *decision threshold* — what evidence level triggers commitment to Adverse vs Disclaimer — not the reasoning that produces the recommendation. Frameworks that tune this threshold explicitly (as plain_python's independence-checker does by demanding citeable support for every claim) will outperform frameworks that optimise reasoning depth. This is consistent with the audit judgement literature's finding that audit failure is rarely a failure of reasoning but frequently a failure of *willingness to act on the reasoning* (Libby, 1995; Han et al., 2011). A similar pattern has been observed in clinical medicine, where diagnostic accuracy is often high but treatment commitment is delayed by thresholds set by incentives other than the evidence; the author notes this parallel without claiming a specific citation, as the clinical-judgement literature on commitment thresholds requires further verification.

5.4 Finding 4 — Agents are systematically over-cautious relative to real auditors — and mostly correctly so

11 of 16 runs produced "Disclaimer" or "Unable". Across all 4 cases, the real auditor issued a clean or unqualified opinion that was indefensible in hindsight. The agents did not replicate this failure: 0 of 16 runs produced "Clean". The over-caution is directionally correct — the structural bias that destroyed Enron, Wirecard, SVB, and Tesco was *under*-caution, driven by client-retention economics that AI agent teams do not face.

But the degree of caution was sometimes excessive. On SVB, the ground truth's `acceptable_ratings` were {Qualified, Adverse} — the evidence (87% uninsured deposits, \$15.1B unrealised HTM losses, 425bp rate hikes, 6.2-year duration mismatch) was sufficient to commit to at least Qualified. Three of four frameworks (crewai, swarm, langgraph) said "Unable" on SVB; only plainpython committed to Qualified. *This is the calibration axis: plainpython scored 4.25, the others 3.50. The difference is associated with plainpython's verdict gate (§5.1), but the deconfounding experiment (§4.5) showed that adding the gate to langgraph did not recover plainpython's calibration (langgraphgate3 scored 2.75 on calibration, worse than langgraph's 3.50). The gate is associated with plainpython's calibration advantage but is not sufficient to transfer it — consistent with Finding 1 (revised).*

Implication for the audit profession. The profession's structural under-caution problem and AI agent teams' structural over-caution problem are mirror images of the same calibration failure: both involve a verdict threshold set by incentives other than evidence sufficiency. For auditors, the incentive is client retention (pushing toward Clean); for AI agent teams, the incentive is the absence of a forcing function (allowing decline-to-opine at no cost). The tuning knob in both

cases is architectural: the profession uses engagement quality review (a verdict gate) to push auditors toward evidence-appropriate opinions; our data suggests AI agent teams need the same structural feature to push them away from excessive caution. The implication for deployment is that AI audit-assist systems should not be deployed without a verdict gate, lest they become high-cost engines of disclaimer.

5.5 Finding 5 — CrewAI's failure was ergonomic, not judgemental

CrewAI scored 0.00 on evidence rigour and reasoning quality across all 4 cases. This is not because CrewAI's agents could not reason — transcripts show extensive multi-turn discussion — but because the sequential Crew process never emitted the `<OPINION>` JSON block the EngagementPartner task specified. The final Task's `expected_output` is a free-form string; CrewAI's abstractions do not expose a hook to enforce a structured output contract on the final task. The team talked for 113–432 seconds per case and produced unparseable prose.

This is an integration failure, not a judgement failure. We cannot rate CrewAI's audit judgement from this run; we can only rate that its output-parsing contract is fragile when the developer needs a machine-readable verdict. The LLM(`model="openai/..."`) routing bug — which cost a full benchmark run to discover and is documented in our AGENTS.md — is a secondary ergonomic finding: CrewAI's LLM wrapper hides provider routing decisions behind a model-name prefix, opaque to developers using a custom OpenAI-compatible endpoint.

Implication for framework selection. For tasks that require structured output (audit opinions, compliance filings, regulatory disclosures), a framework's *output contract enforcement* matters more than its orchestration model. `plain_python`'s `<OPINION>` tag parsing is crude (a regex) but it is *enforced* — the independence-checker rejects drafts that lack it. CrewAI's elegant agent abstractions provided no equivalent pressure point. Framework selection for professional-judgement tasks should weight output-contract enforceability alongside orchestration sophistication.

5.6 Finding 6 — Latency varies ~5.5× across frameworks and is uncorrelated with quality

Framework	Mean s/case	Quality rank
swarm	53.6	3rd
langgraph	271.6	2nd
plain_python	292.2	1st
crewai	295.8	4th

Mean-to-mean latency ratio is 5.5× ($\text{crewai/swarm} = 295.8/53.6$); an earlier draft of this paper reported "9×", which was a factual error (no pair of mean latencies yields 9×) identified by the peer review (§3.4) and corrected here. Swarm was 5× faster than the next framework and produced the worst judgement accuracy. `langgraph`'s Enron run (693s) included 2 skeptic revisits — the most expensive single run — and still produced a wrong rating. Latency buys more LLM

calls, not better verdicts. The cheapest framework had the highest recall; the most expensive runs did not convert extra calls into rating flips.

Implication for deployment. In production audit-assist systems where cost matters, *plainpython* (~292s, 1st in quality) and *langgraph* (~272s, 2nd in quality) have similar mean latency ($\Delta=20.6s$, 7.6% — within noise), so the quality difference is not bought with extra latency. The sophisticated loop costs the same and buys nothing on this workload. Note: an earlier draft claimed *plainpython* "dominates" *langgraph* on price-performance; this was directionally wrong (*plain_python* is 7.6% slower, not faster) and has been corrected to "similar cost, higher quality". This is consistent with the audit judgement literature's finding that additional review cycles without a forcing function do not improve opinion quality (Schneider and Messier, 2007, on the difference between "more review" and "effective engagement quality review").

5.7 Finding 7 — The framework's control flow should match the task's dominant failure mode

Findings 1–6 are consistent with a single matching principle: framework architecture should be selected based on the task's dominant failure mode, not on framework sophistication. On the audit-judgement task in this benchmark, the dominant failure mode was **under-commitment** — the model reasons correctly (5/5 from the LLM judge on 7 of 16 runs, including runs whose verdict was wrong) and then refuses to opine (9 of 16 runs said "Unable"). The architectural features that helped were the ones that *removed* the agent's choice to defer: the fixed cook order (*plain_python*), the forced <OPINION> JSON output contract, the rejection-with-cap loop that prevents infinite rework but does not allow infinite refusal. The architectural features that hurt were the ones that *added* choice: Swarm's handoff chain (the agent can always hand off instead of committing — it did, on 4 of 4 cases), CrewAI's free-form final-task output (the agent can produce prose that never converges to a verdict — it did, on 4 of 4 cases), LangGraph's optional revisit loop (the agent can defer to another round — the loop fired on 3 of 4 cases and never flipped a verdict).

This pattern predicts a complementary result on a different task. On tasks where the dominant failure mode is **under-exploration** — coding (the agent writes the first version, doesn't test edge cases), research (the agent synthesises the first source, doesn't search further), debugging (the agent patches the symptom, doesn't look for the root cause) — the matching principle predicts that agentic features (autonomy, tool-calling, iterative handoffs) should outperform scripted control flow, because the agent needs to *do more*, not *commit harder*. A script that forces commitment on a coding task would produce bad code that compiles on the first try; an agent that iterates would produce better code. The two task types have opposite failure modes and therefore opposite optimal architectures.

The audit profession's own standards already implement this matching principle, though it has not, to our knowledge, been articulated as a design rule for agent systems. ISA 240 (fraud) mandates brainstorming sessions — an exploratory, agentic step where the team generates hypotheses without commitment. ISA 700 (opinion) mandates a fixed opinion format with EQR

sign-off — a scripted, forced-commitment step where the team must emit one of five verdicts or say why it cannot. The profession matches the control flow to the failure mode: explore when the risk is missing something, commit when the risk is refusing to decide. Agent framework designers have not yet learned this — they build one architecture (autonomous agents with handoffs) and apply it to all tasks, including judgement tasks where autonomy is the failure mode, not the cure.

Implication for framework design and selection. The matching principle is the generalizable claim that survives the $n=4$ limitation of this benchmark because it is a *principle* (match architecture to failure mode), not a *ranking* (framework X is best). It predicts that the `plain_python > langgraph > swarm > crewai` ordering observed here will reverse on exploration-dominant tasks, and that no single framework architecture will dominate across task types. If this prediction is confirmed by future work on coding/research benchmarks with the same frameworks and personas, the matching principle becomes a design rule: *select the framework whose control flow imposes the constraint that counters the task's dominant failure mode*. For judgement tasks, that is a scripted gate; for exploration tasks, that is agentic autonomy. Using an agentic framework for a judgement task gives the agent more ways to avoid the thing it is already bad at doing — and our data show it will use every one of them.

6. Limitations

12. **Sample size (n=4 cases).** Differences of ≤ 0.5 on any axis should not be treated as statistically significant. The 5-axis profile is more informative than the total. A follow-up with 20–30 cases would permit per-axis significance testing.
13. **Single LLM.** All runs used glm-5.2:cloud. Framework effects may interact with model effects — a framework that helps a weaker model might not help a stronger one. A follow-up with 2–3 models (including a non-reasoning model) would isolate this.
14. **Single LLM judge.** Reasoning quality was scored by glm-5.2:cloud, the same model that ran the agent teams. Known self-preference risk. Mitigated by a tight rubric and single-call design, but a follow-up with a different judge model (or human scoring on a sample) would strengthen the claim.
15. **Curated evidence.** Agent-visible evidence is a subset of pre-scandal public information, selected by the authors. A real auditor would have had working papers, management access, and prior-year context. The benchmark tests judgement under evidence constraint, not full audit conditions. Different evidence curation could change the calibration axis meaningfully.
16. **Ground truth is a set, not a point.** Audit opinions are not uniquely determined by the facts; we score against `acceptable_ratings` (a set of 2 per case). This is methodologically defensible but allows "adjacent" ratings that a stricter rubric would mark wrong.
17. **CrewAI's failure mode.** We scored CrewAI on the output it produced (unparseable prose → Unable with 0 red flags). A different output-contract implementation (e.g., a custom Crew

output parser, or a final Task with a Pydantic expected_output schema) might recover CrewAI's actual judgement capability. Our finding is best read as "CrewAI as configured by our implementation failed", not "CrewAI cannot do audit judgement".

18. **No human baseline.** We compare AI teams to ground truth but not to human auditors working the same evidence under the same constraint. A human baseline would calibrate whether the AI teams' over-caution is excessive relative to what a competent human would do with the same evidence.
 19. **Sequential calls, no parallelism.** Real audit teams work in parallel; our sequential design isolates framework effects but may disadvantage frameworks whose orchestration model assumes parallelism (e.g., LangGraph's design supports concurrent fan-out, which we did not exercise).
 20. **Scoring artefact on reasoning quality.** The scorer uses a sentinel value of -1 when the judge LLM call itself fails (one case: swarm/Wirecard). This -1 is included in the headline mean (swarm 3.50 = mean of {5, 5, 5, -1}), understating swarm's true reasoning quality (5.00 on 3 of 4 cases). A scoring fix that excludes failed judge calls from the mean would correct this but would reduce n for swarm's reasoning axis to 3. The artefact is flagged in §4.1 and retained for transparency.
 21. **Deconfounding runs are n=1 per cell.** The §4.5 deconfounding experiment added 8 runs (*langgraphgate* × 4, *langgraphgate3* × 4) but each is still n=1 per cell. The 2.31-point gap between *langgraphgate3* and *plainpython* is within plausible run-to-run LLM variance (see Limitation 1). A multi-run deconfounding design (3 runs per cell, n=12 per framework) would settle whether the gap is signal or noise.
 22. **Peer review by the same model.** The peer-review system (§3.4) used glm-5.2:cloud — the same model that wrote the paper and ran the agent teams. The peer review's correct identification of the gate-framework confound may reflect the model's training-data familiarity with the audit-judgement literature rather than genuine analytical independence. A peer review by a different model would strengthen the methodological claim that the peer-review process is itself a valid experimental tool.
 23. **Famous-case training-data contamination.** All four cases (Enron, Wirecard, SVB, Tesco) are among the most documented scandals in history; the model's training data almost certainly contains detailed analyses. The forward-reasoning claim (agents reason from evidence, not pattern-match) is not fully testable on famous cases without controls for training-data contamination. An anonymised-case variant (same evidence, redacted company names) would test this.
-

7. Conclusion and future work

We benchmarked four multi-agent frameworks on a shared audit-judgement task: four real scandal cases, four canonical audit personas, a four-axis scorer separating evidence-surfacing from opinion-formation. The lowest-abstraction framework (plain Python with a Blackboard

pattern) outperformed three production frameworks. We initially hypothesised the differentiator was a *forced verdict gate* (independence-checker rejection loop); a deconfounding experiment adding the gate to LangGraph did not confirm this, and the leading alternative explanation is the Blackboard pattern's input-digest reactivity — but we cannot distinguish this from run-to-run variance at $n=1$ per cell. Evidence-surfacing and opinion-formation appear decoupled (Swarm: highest recall, lowest accuracy), though the cross-framework correlation is positive, not negative as an earlier draft claimed. Agents were systematically over-cautious relative to the real auditors who failed (0 of 16 runs issued "Clean"), and mostly correctly so given the evidence constraint — but the degree of caution was sometimes excessive. The paper was independently peer-reviewed by a second LangGraph multi-agent system, whose reviewers correctly identified the gate-framework confound that the deconfounding experiment confirmed; this peer-review process is itself part of the methodology and a candidate for broader adoption in LLM-agent research.

Four directions for future work:

24. **Scale and multi-run.** Extend to 20–30 cases spanning more jurisdictions, audit firms, and failure types; run 3 replicates per cell ($n=12$ per framework) to settle whether the *plainpython vs langgraphgate3* gap is signal or noise. Include non-scandal control cases where Clean is the correct opinion, to test whether AI agents issue Clean when appropriate.
25. **Model sweep.** Run the same benchmark with 2–3 LLMs (a reasoning model, a fast model, a non-reasoning model) to separate framework effects from model effects. Our single-model design isolates framework effects but cannot detect interactions.
26. **Human baseline.** Recruit 4–8 senior auditors to perform the same task on the same evidence. This would calibrate whether AI teams' over-caution is excessive relative to what a competent human would do with the same evidence, and situate the benchmark within the audit judgement literature's existing baselines.
27. **State-propagation deconfounding.** Add the partner's prior draft to *langgraphgate3's senior node state field* (so the senior sees the same information *plainpython's* blackboard exposes automatically). If this closes the gap, the Blackboard's input-digest reactivity is confirmed as the driver; if not, the result is more likely noise or another unidentified mechanism.

The broader claim, if the deconfounding result replicates at scale, is more nuanced than our initial hypothesis: the audit profession's structural under-caution problem and AI agent teams' structural over-caution problem are the same problem at different thresholds, but the architectural fix is not as simple as "add a verdict gate". The *state propagation mechanism* — how upstream writes reach downstream agents on rejection-revisit — may be the load-bearing structural feature, not the gate itself. Engagement quality review is not a bureaucratic layer; it is the profession's existing verdict gate, but its effectiveness depends on *how* it is wired into the firm's information flow, not merely *whether* it exists. Framework designers who omit it are building audit teams that will either rubber-stamp (the human failure mode) or refuse to opine (the AI failure mode we observed). Neither is acceptable for a profession whose product is opinion.

References

- Butt, J. L., & Campbell, T. L. (1989). The effects of information order and hypothesis-testing strategies on auditors' judgments. *Accounting, Organizations and Society*, 14(5-6), 471–479.
- Carey, P., & Simnett, R. (2006). Audit partner tenure and audit quality. *The Accounting Review*, 81(3), 653–676.
- Corkill, D. D. (1991). Blackboard systems. *AI Expert*, 6(9), 40–47.
- CrewAI. (2025). *CrewAI documentation*. Retrieved from <https://docs.crewai.com>
- DeAngelo, L. E. (1981). Auditor size and audit quality. *Journal of Accounting and Economics*, 3(3), 183–199.
- Epps, K. K., & Messier, W. F. (2007). Engagement quality reviews: A comparison of audit firm practices. *Auditing: A Journal of Practice & Theory*, 26(2), 167–181.
- Han, J., Jamal, K., & Tan, H. T. (2011). Auditors' overconfidence in predicting the technical knowledge of superiors and subordinates. *Auditing: A Journal of Practice & Theory*, 30(3), 129–151.
- International Auditing and Assurance Standards Board (IAASB). (2024). *International Standards on Auditing (ISA) 220, 240, 500, 530, 700, 705*. International Federation of Accountants. Retrieved from <https://www.iaasb.org/standards>
- Jimenez, C. E., Yang, S., Mu, D., Sachdev, A., & Arora, R. (2023). SWE-bench: Can language models resolve real-world GitHub issues? arXiv:2310.06770.
- Johnson, V. E., Khurana, I. K., & Reynolds, J. K. (2002). Audit-firm tenure and the quality of financial reports. *Contemporary Accounting Research*, 19(4), 637–660.
- LangChain. (2025). *LangGraph documentation*. Retrieved from <https://langchain-ai.github.io/langgraph>
- Libby, R. (1995). The role of knowledge and memory in audit judgment. In A. H. Ashton & R. H. Ashton (Eds.), *Judgment and Decision-Making Research in Accounting and Auditing* (pp. 176–206). Cambridge University Press.
- Liu, X., Yu, C., Yu, H., Tang, Q., Zhao, D., Wang, J., & Zhang, Y. (2023). AgentBench: Evaluating LLMs as agents. arXiv:2308.03688.
- Mialon, G., Mialon, R., Huang, S., & Peng, X. (2023). GAIA: A benchmark for general AI assistants. arXiv:2311.12983.

Nii, H. P. (1986). The blackboard model of problem solving and the evolution of blackboard architectures. *AI Magazine*, 7(2), 38–53 (Part 1); 7(3), 82–106 (Part 2).

OpenAI. (2024). *Swarm: Educational framework for multi-agent orchestration*. GitHub repository. Retrieved from <https://github.com/openai/swarm>

Public Company Accounting Oversight Board (PCAOB). (2024). *Auditing Standard AS 1210: Engagement Quality Review*. PCAOB. Retrieved from <https://pcaobus.org/standards/auditing-standards>

Schreyer, M., Sattarov, T., Borth, D., Dengel, A., & Reimer, B. (2018). Detection of anomalies in large-scale accounting data using deep autoencoder neural networks. In *ICML 2018 Workshop on Accounting Data Analytics*. arXiv:1809.08535.

Schneider, A., & Messier, W. F. (2007). Engagement quality review: Insights from the academic literature. *Managerial Auditing Journal*, 22(8), 828–843.

Watts, R. L., & Zimmerman, J. L. (1983). Agency problems, auditing, and the theory of the firm: Some evidence. *The Journal of Law and Economics*, 26(3), 613–633.

Appendix A: Reproducing the benchmark

The benchmark code and all raw data are in `audit_bench/` of the experiment repository. To reproduce:

```
# Phase 0 (optional): verify tool-calling support
.\.venv\Scripts\python.exe audit_bench\_probe_tool_call.py

# Full benchmark (4 fw x 4 cases x 1 model, ~64 min wall-clock)
.\.venv\Scripts\python.exe audit_bench\run_benchmark.py --frameworks all
--cases all --models glm-5.2:cloud

# Generate the rating report
.\.venv\Scripts\python.exe audit_bench\rate_frameworks.py
audit_bench\runs\bench_<ts>\all_scores.json
```

The timestamped research log at `audit_bench/runs/bench_<ts>/research_log.md` records every LLM call across all 16 runs with elapsed times. Per-run transcripts are in `raw/<framework>/<model>/<case>/result.json`.

Appendix B: Framework implementation LOC

Framework	LOC	Notes
-----------	-----	-------

crewai	111	Lightest, but most behaviour is implicit in Agent/Task/Crew abstractions
plain_python	112	Reuses proven Blackboard pattern; explicit loop control
swarm	133	Native tool-calling; handoff-driven
langgraph	177	Heaviest; state schema + conditional edge + LangChain wrapper boilerplate